German Credit Risk Classification

Machine Learning Project 1 Report - Spring Semester 2026

Kinga Kerekes

German Credit Risk Classification – Machine Learning Project 1 Report

Spring Semester 2026 – Kinga Kerekes

Table of Contents

1. Exploratory Data Analysis 3
   Dataset Overview 3
   Class Distribution 3
   Missing Values 3
   Feature Summary 3
   Feature Distributions by Class 4
   Categorical Feature Analysis 5
   Preprocessing Implications 5
2. Preprocessing 6
   Train/Test Split 6
   Feature Types 6
   Standardization of Numerical Features 6
   One-Hot Encoding of Categorical Features 6
   Pipeline Structure 6
3. Choice of Evaluation Metric: F1-Score 7
   Why Not Accuracy, Recall or Precision alone? 7
   The F1-score 7
   F1 vs. AUC 7
4. Model Selection: The Perceptron 8
   Assumptions, Convergence and Suitability 8
5. Linear Classification Models 9
   Logistic Regression 9
   Linear Support Vector Machines 10
   Why Both Models Have Moderate F1 10
6. Non-Parametric Model: k-Nearest Neighbors 12
   Results (k=5, best valid k) 12
   k=1: Overfitting and Why It Was Excluded 13
   Bias-Variance Analysis 13
7. Comparative Analysis and Discussion 14
   Summary of Results 14
   Why Logistic Regression and Linear SVM Give Different Results 15
   Probability Estimates and Their Relevance to F1 15
   Model Recommendation 15
8. Ways to Improve the Models 16
   Why F1 Guides the Improvement Strategy 16
   The Cost Structure of the Problem 16
   Implementing Cost-Sensitive Weights 16
   Results After Improvement 17

1. Exploratory Data Analysis

Dataset Overview

The German Credit dataset contains 1,000 observations and 20 features, comprising a mix of numerical and categorical variables. The target variable (y) indicates credit risk, encoded as good (0) and bad (1), showing whether a person with 20 different attributes has a low or a high credit risk. Our objective is to train a model that is able to predict on the historical data whether a new person will be high or low risk (binary classification).

Class Distribution

The dataset is moderately imbalanced: 700 applicants (70%) are at a good risk (likely to pay back) and 300 (30%) are at a bad risk (not likely to pay back). This imbalance makes accuracy unreliable and motivates the use of F1-score as the primary metric.

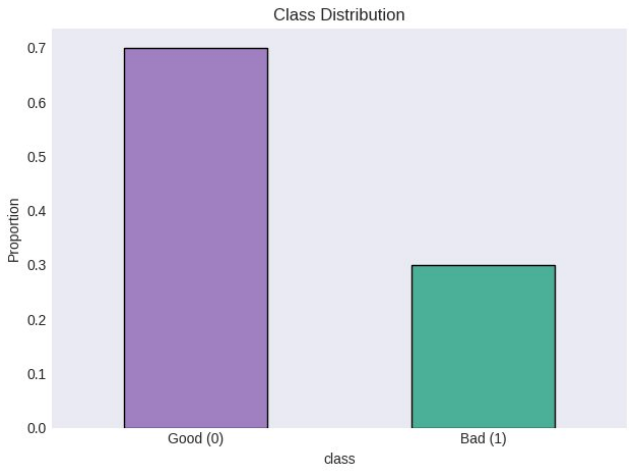


Figure 1. Class distribution of the target variable (Good = 70%, Bad = 30%).

Missing Values

No missing values were found in any of the 20 features or in the target variable. No imputation was required.

Feature Summary

The dataset contains 7 numerical features. Key observations:

- Duration ranges from 4 to 72 months, with a mean of ~21 months and high variability (std ≈ 12)
- Credit amount ranges widely from 250 to 18,424, with a mean of ~3,271
- Age ranges from 19 to 75, with a mean of ~36 years
- Features such as: installment_commitment, residence_since, existing_credits and num_dependents do not present big variations (1-4 range)

Feature	Mean	Std	Min	Max
duration	20.90	12.06	4	72
credit_amount	3271.26	2822.74	250	18424
installment_commitment	2.97	1.12	1	4
residence_since	2.85	1.10	1	4
age	35.55	11.38	19	75
existing_credits	1.41	0.58	1	4
num_dependents	1.16	0.36	1	2

Since numerical features vary widely in scale, (e.g.: credit_amount ranges up to 18,424 while num_dependents only takes values 1 or 2), we would need to implement some sort of standardization. Without it, large-scale features would dominate distance calculations in kNN and gradient updates in Logistic Regression and SVM, regardless of their actual predictive importance.

Feature Distributions by Class

The histograms reveal that good and bad applicants overlap substantially across all numeric features, confirming that the classes are not linearly separable. Bad applicants tend to have longer loan durations and higher credit amounts. Features like existing_credits and num_dependents show almost no separation between classes, suggesting they will contribute little to performance.

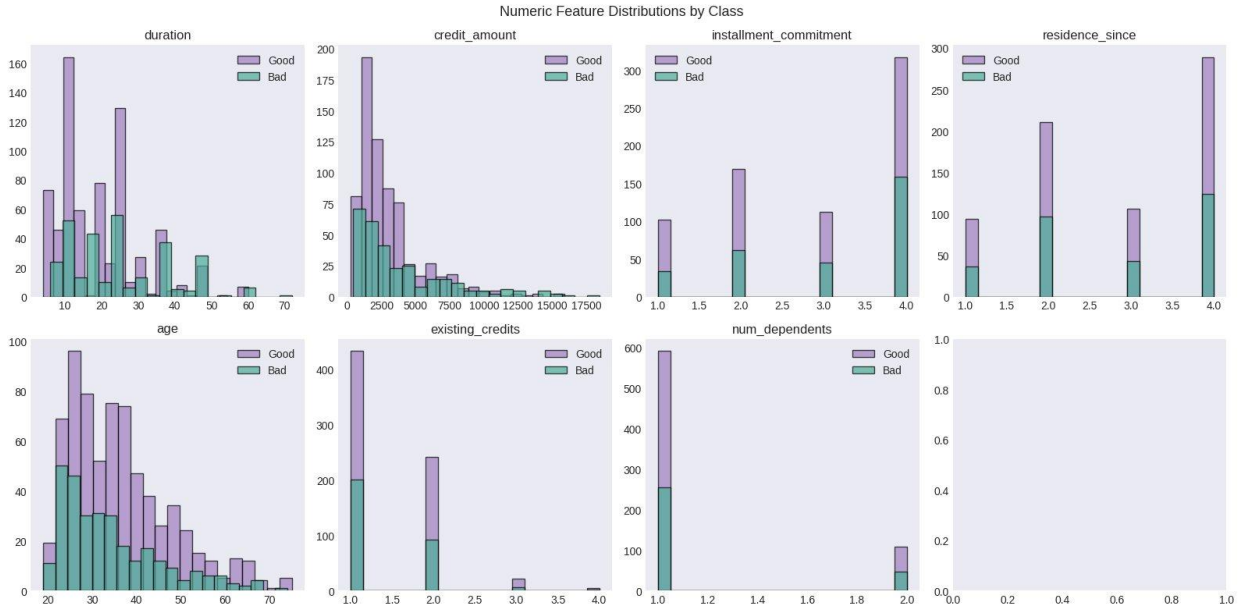


Figure 2. Numeric feature distributions by class (Good = purple, Bad = teal).

The boxplots confirm these observations. Bad applicants have notably higher medians for both duration (~24 months vs. ~18) and credit amount. The age distributions are largely overlapping.

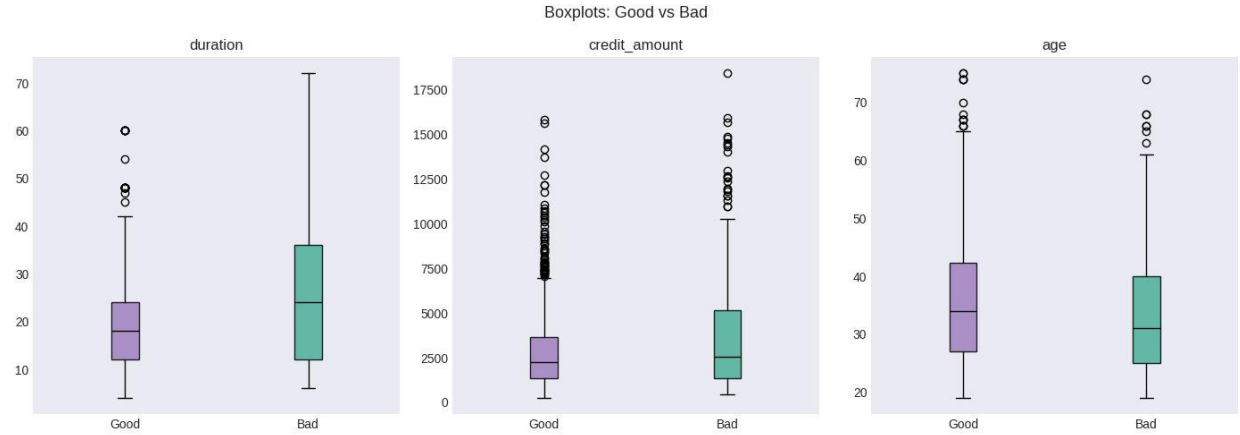


Figure 3. Boxplots of key numeric features by class.

Categorical Feature Analysis

Examining, for instance credit_history against the target reveals a strong relationship. Applicants with "critical/other existing credit" are bad risks over 80% of the time, while those with "no credits/all paid" are predominantly good risks (~62%).



Figure 4. Good vs. bad risk rate by credit history category.

Preprocessing Implications

The substantial class overlap observed across all features suggests that no model will achieve very high F1 on this dataset, since the inherent difficulty of separating the classes sets an upper bound on performance. The EDA also motivates standardization of numerical features and one-hot encoding of categorical features as necessary preprocessing steps.

2. Preprocessing

Train/Test Split

The dataset was split into a training set (80%, 800 samples) and a test set (20%, 200 samples) using stratified sampling. Stratification ensures that the same 70/30 class ratio is preserved in both splits as observed in the overall dataset, which is important for reliable F1 evaluation. The test set was kept completely untouched until final model evaluation.

Feature Types

The dataset contains two types of features requiring different transformations:

- 7 numerical features: duration, credit_amount, installment_commitment, residence_since, age, existing_credits, num_dependents
- 13 categorical features: checking_status, credit_history, purpose, savings_status, employment, personal_status, other_parties, property_magnitude, other_payment_plans, housing, job, own_telephone, foreign_worker

Standardization of Numerical Features

Numerical features were standardized using StandardScaler (z-score normalization). As mentioned above, without standardization, features with large ranges (e.g. credit_amount) would dominate those with small ranges (e.g. num_dependents, values 1–2), particularly harming kNN where distances drive all predictions.

One-Hot Encoding of Categorical Features

One-hot encoding converts each categorical feature into a set of binary columns**,** one for each possible category. For a given observation, exactly one of those columns is set to 1 (indicating which category applies) and all others are set to 0.

Pipeline Structure

All preprocessing steps were combined with the classifier inside a single sklearn Pipeline using a ColumnTransformer. The pipeline is fitted only on the training data, meaning the scaler's mean and standard deviation and the encoder's category mappings are learned exclusively from training samples.

3. Choice of Evaluation Metric: F1-Score

Why Not Accuracy, Recall or Precision alone?

The German Credit dataset is imbalanced: 70% of applicants are classified as good risk and 30% as bad risk. In this setting, accuracy is a misleading metric. A classifier that always predicts "good" would achieve 70% accuracy without learning anything at all. Accuracy rewards predicting the majority class and does not distinguish between the two types of error, which might have different consequences in a credit context.

This motivates turning to precision and recall; metrics that focus specifically on the minority class. However, neither extreme is useful on its own either. A model that flags every applicant as bad risk achieves perfect recall but is completely unusable, since it would reject every customer including creditworthy ones. A model that only predicts bad risk when it is certain, achieves high precision but misses most defaulters, defeating the purpose of the classifier. What is needed is a metric that rewards models that are both reasonably good at catching bad-risk applicants and reasonably careful about not raising too many false alarms. F1-score achieves this by combining precision and recall into a single score through the harmonic mean, making it the appropriate choice for this task.

The F1-score

The F1-score is the harmonic mean of precision and recall:

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

It is particularly well-suited to this dataset for three reasons. First, it is robust to class imbalance: unlike accuracy, it focuses more on the minority class (bad risk) and is not impacted by the model's performance on the majority class that much. Second, it penalizes imbalanced precision-recall tradeoffs, because it uses the harmonic mean: a model with very high precision but very low recall (or vice versa) will score poorly, pushing models toward a balanced performance. Third, it is directly interpretable: an F1 of 0.63 means the model is simultaneously reasonably precise and reasonably sensitive in identifying bad-risk applicants.

This is why F1-score was used as the scoring metric in GridSearchCV for hyperparameter tuning across all models, and why it is the primary metric for comparing model performance throughout this report. Where relevant, recall is discussed as a secondary metric, since in a real credit context, missing a bad-risk applicant (false negative) is typically more costly than a false alarm (false positive).

It should be noted however that when the primary concern is minimizing the number of missed bad-risk applicants, recall becomes the more important component of F1, and improvements that boost recall at the cost of some precision are considered worthwhile as long as the overall F1 score increases or stays the same. F1 is therefore used as the primary metric for model comparison and hyperparameter tuning, while Section 8 addresses the cost asymmetry directly by incorporating domain-informed weights into the models themselves.

F1 vs. AUC

AUC (Area Under the ROC Curve) is another strong metric for imbalanced datasets and is reported in comparative analysis. The key difference is that AUC is threshold-independent, since it measures overall discriminative ability across all possible thresholds, while F1 is evaluated at the default threshold of 0.5. For model selection and tuning, F1 is more direct, since it reflects actual deployed performance. AUC is useful as a complementary summary of how much overall separation the model achieves.

4. Model Selection: The Perceptron

Assumptions, Convergence and Suitability

The Perceptron assumes that the data is linearly separable, meaning that a single hyperplane can divide the dataset into two classes.

Convergence is guaranteed if and only if the data is linearly separable. In that case, the algorithm makes at most $(\frac{R}{\gamma})^2$ mistakes, where R is the maximum norm of any input vector and γ is the margin of the dataset.

The Perceptron is probably not appropriate for this dataset. Real-world credit data is unlikely to be linearly separable, since good and bad applicants can overlap significantly across features (as also seen in the EDA). If the data is not linearly separable, the Perceptron never converges and cycles indefinitely.

5. Linear Classification Models

Both models were trained inside a Pipeline with the ColumnTransformer preprocessor. The regularization parameter C was tuned via 5-fold stratified cross-validation using GridSearchCV with F1-score as the scoring metric.

Logistic Regression

Logistic Regression is one of the linear models implemented to predict the probability of an applicant being a bad credit risk. It assigns a weight to each feature and combines them linearly, then uses a sigmoid function to convert results into a probability between 0 and 1 (with 0.5 as a threshold to determine whether an applicant is good or bad risk). The parameter C controls how strongly the model is regularized, as a lower C prevents overfitting by keeping the weights small.

The best C found by the model was 0.6158. Results on the test set:

Metric	Value
Train Accuracy	0.7837
Test Accuracy	0.7850
Precision	0.6809
Recall	0.5333
F1-Score	0.5981
AUC	0.81

The F1-score of 0.598 reflects a conservative model: it correctly identifies 32 out of 60 bad-risk applicants (recall = 0.533) with relatively high precision (0.681). The model predicts “good” more often than “bad”, which keeps false positives low but leaves many true bad-risk cases undetected, this way pulling recall, and therefore F1, down.

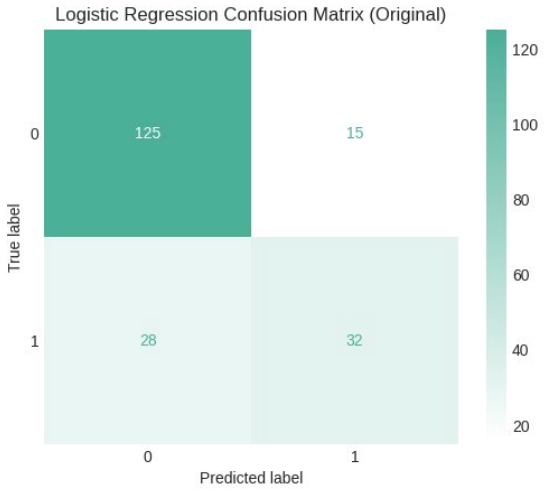


Figure 5. Logistic Regression confusion matrix.

Linear Support Vector Machines

Linear SVM aims to find a straight decision boundary that separates the two classes with the largest possible gap (margin). The training points closest to the boundary are called support vectors, and they are the only ones that determine where the boundary is drawn. Like Logistic Regression it is a linear model, but instead of minimizing a probabilistic loss it focuses on maximizing this margin. The parameter C controls how strictly the boundary must separate the classes.

The best C found was also 0.6158. Results on the test set:

Metric	Value
Train Accuracy	0.7875
Test Accuracy	0.7750
Precision	0.6415
Recall	0.5667
F1-Score	0.6018
AUC	0.80

The SVM achieves a marginally higher F1 of 0.602, primarily through better recall (0.567 vs. 0.533): it catches 34 bad-risk applicants compared to LR's 32. The tradeoff is slightly lower precision (0.6415 vs. 0.6809). In F1 terms the difference is negligible (0.004), but the recall advantage is meaningful from a credit risk perspective.



Figure 6. Linear SVM confusion matrix.

Why Both Models Have Moderate F1

Both models achieve F1 around 0.60, which falls in the moderate range. This is expected and not a sign of implementation error. The substantial class overlap seen in the EDA sets a natural ceiling on how well any

linear model can separate the classes. Good and bad applicants share similar distributions across most features, meaning no linear boundary can cleanly divide them.

The train-test gap is small for both models, Logistic Regression (0.784 train vs. 0.785 test) and Linear SVM (0.788 train vs. 0.775 test), which confirms that neither model is overfitting. The model has not simply memorized the training data but has learned patterns that generalize to unseen applicants.

The main limitation of both baseline models is low recall: Logistic Regression identifies only 32 out of 60 bad-risk applicants, and Linear SVM identifies 34. This means both models miss roughly half of all defaulters at the default threshold of 0.5, which is the primary driver of the moderate F1 scores. Addressing this directly, rather than further tuning C, is therefore the most promising path to improvement, as explored in Section 8.

6. Non-Parametric Model: k-Nearest Neighbors

k-Nearest Neighbors is a more intuitive classifier. To predict whether a new applicant is a bad risk, it looks at the k most similar applicants in the training data and takes a majority vote. Similarity is measured by distance in feature space, which is why scaling is important: without it, features with large values would dominate the distance calculation.

The kNN classifier was trained using the same Pipeline. The number of neighbors k was tuned via 5-fold stratified cross-validation (F1 scoring) over odd values from 3 to 29, since these are preferred for binary classification to avoid tied votes.

The upper bound for k follows the common practice, taking the root of the number of training samples $\sqrt{number\ of\ training\ data} \approx \sqrt{800} \approx 28.3$. K=1 was excluded from tuning and evaluated separately.

Results (k=5, best valid k)

Metric	Value
Train Accuracy	0.8212
Test Accuracy	0.7250
Precision	0.5676
Recall	0.3500
F1-Score	0.4330
AUC	0.73

The F1-score of 0.433 is substantially lower than both linear models. The recall of 0.350 is the main driver of this result: kNN misses 39 out of 60 bad-risk applicants. This is the worst F1 of all models tested and makes kNN unsuitable for this task.

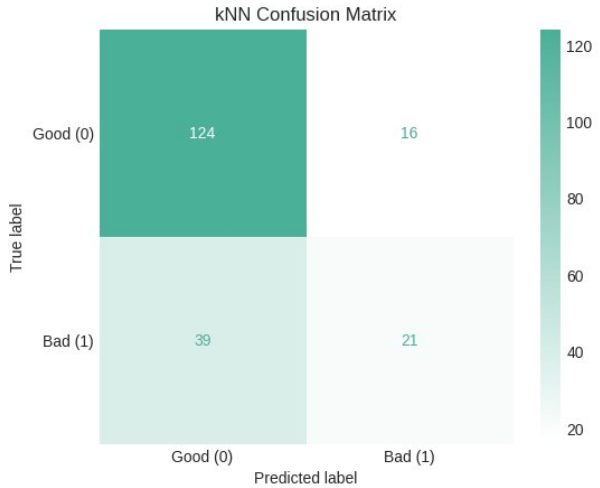


Figure 7. kNN confusion matrix (k=5).

k=1: Overfitting and Why It Was Excluded

With k=1, every training points' nearest neighbor is itself, yielding a training accuracy of 1.000, which is pure memorization. Although the test F1 of 0.526 appears better than k=5's 0.433, this apparent advantage is not genuine generalization. The CV F1 (Cross-Validation F1) for k=1 (shown as the purple dot in Figure 9) is 0.456, which is only slightly above k=3's CV F1. The test result of 0.526 is probably a result of the specific test split, not a reliable estimate of performance.

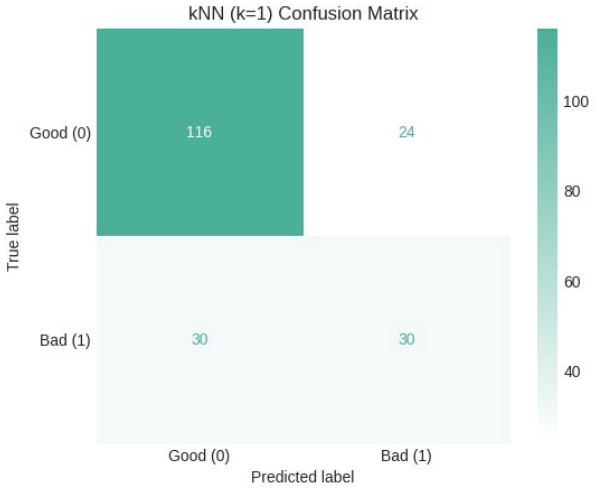


Figure 8. kNN k=1 confusion matrix.

Bias-Variance Analysis

The bias-variance plot shows CV F1-score decreasing from k=3 to k=29, with a slight peak at k=5. The absence of a U-shape confirms there is no optimal k in this range, and increasing k consistently increases bias faster than it reduces variance. This behavior is caused by the high-dimensional sparse feature space created by one-hot encoding: in high dimensions, all points tend to become equidistant, making neighborhood structure unreliable and dragging F1 down regardless of k.

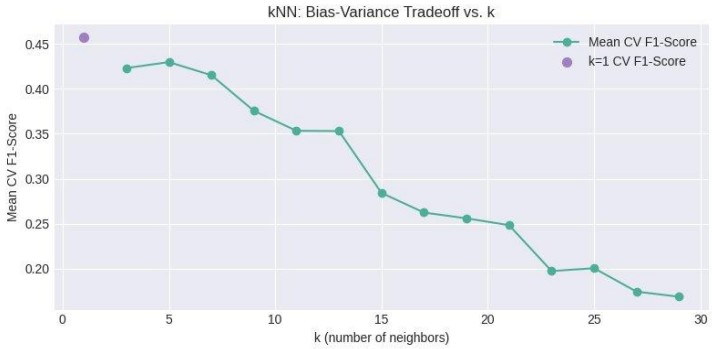


Figure 9. CV F1-score vs. k. purple dot = k=1 CV F1 (0.456).

7. Comparative Analysis and Discussion

Summary of Results

Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-Score	AUC
Logistic Regression	0.784	0.785	0.681	0.533	0.598	0.81
Linear SVM	0.788	0.775	0.642	0.567	0.602	0.80
kNN (k=5)	0.821	0.725	0.568	0.350	0.433	0.73
kNN (k=1)	1.000	0.730	0.556	0.500	0.526	0.66

Ranked by F1-score: Linear SVM (0.602) ≈ Logistic Regression (0.598) >> kNN k=1 (0.526) > kNN k=5 (0.433). The two linear models are essentially tied on F1, with a difference of only 0.004, though SVM has a slight recall advantage (0.567 vs. 0.533), meaning it catches 2 more bad-risk applicants out of 60. Logistic Regression compensates with higher precision (0.681 vs. 0.642) and a marginally better AUC (0.81 vs. 0.80), suggesting slightly better overall discriminative ability.

Both kNN variants perform substantially worse on F1, confirming that distance-based methods are not well suited to this dataset. Notably, kNN k=1 achieves a higher test F1 (0.526) than k=5 (0.433), but this should not be interpreted as genuine superiority. Its training accuracy of 1.000 reveals pure memorization, and its AUC of 0.66 is the lowest of all models, indicating poor overall ranking ability. kNN k=5 is the more honest result: it avoids overfitting but still achieves the worst F1 and recall (0.350) of all models, missing 39 out of 60 bad-risk applicants.

The train-test gap for the linear models is negligible (LR: 0.784 vs. 0.785, SVM: 0.788 vs. 0.775), confirming no overfitting. For kNN k=5 the gap is small but more visible (0.821 vs. 0.725), suggesting some remaining sensitivity to the training data even at k=5.

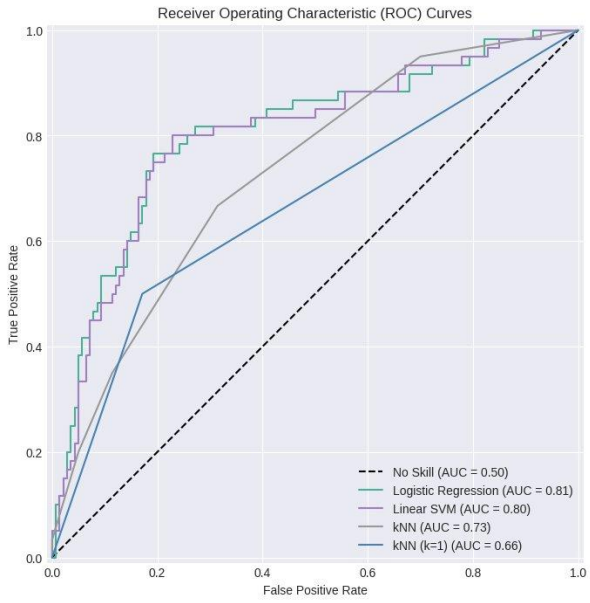


Figure 10. ROC curves for all models. LR (AUC=0.81) and SVM (AUC=0.80) outperform both kNN variants on AUC, consistent with the F1 ranking.

Why Logistic Regression and Linear SVM Give Different Results

Both models learn a linear decision boundary but via different objectives. Logistic Regression minimizes cross-entropy loss across all training points, while SVM maximizes the geometric margin and focuses only on the support vectors nearest the boundary. This means SVM is less influenced by easy-to-classify samples and more focused on the hard cases, which may explain why it is slightly better in both recall and F1.

Probability Estimates and Their Relevance to F1

Logistic Regression outputs a probability for each applicant rather than a hard yes/no decision. This is directly relevant to F1: by adjusting the decision threshold away from the default 0.5, the precision-recall trade-off can be shifted to find the point where F1 is maximized. Linear SVM only outputs a hard class decision, so this kind of threshold tuning is not directly available. kNN can produce rough probability estimates from the proportion of bad-risk neighbors, but these are less reliable. Among the three models, Logistic Regression therefore offers the most control over F1 optimization.

Model Recommendation

Based on F1-score, Linear SVM (0.602) is the recommended model for this task, edging out Logistic Regression (0.598). While the F1 difference is small (0.004), SVM also has a recall advantage (0.567 vs. 0.533), since it catches 2 more bad-risk applicants out of 60. Since this report prioritizes F1 as the primary metric, and recall is identified as the more important component given the cost structure of the problem, SVM is the more appropriate choice at the default threshold.

Logistic Regression remains a strong alternative and has advantages in other areas: it achieves a marginally higher AUC (0.81 vs. 0.80), its calibrated probability outputs allow threshold tuning to further optimize F1 in production, and its coefficients are directly interpretable, which might be important for regulatory compliance in financial applications. If the bank requires an explainable model or wants flexibility over the decision threshold, Logistic Regression would be the preferred choice.

In summary, Linear SVM is recommended based on F1 and recall performance, while Logistic Regression is the stronger choice if interpretability and probability calibration are priorities. The final choice between the two depends on the specific operational requirements of the institution.

8. Ways to Improve the Models

Why F1 Guides the Improvement Strategy

The baseline linear models achieve F1 around 0.60, held back primarily by low recall (both miss roughly half of bad-risk applicants). Since F1 is the harmonic mean of precision and recall, improving recall (even at some cost to precision) will improve F1 as long as the recall gain is proportionally larger than the precision loss. This motivates addressing the cost asymmetry between the two types of misclassifications directly.

The Cost Structure of the Problem

The German Credit dataset carries an explicit domain-defined cost structure. According to the dataset documentation on OpenML:

> *"It is worse to class a customer as good when they are bad (5), than it is to class a customer as bad when they are good (1)."*
>
> *— OpenML, German Credit Dataset (ID 31), https://www.openml.org/search?type=data&sort=runs&id=31&status=active*

This means a false negative, approving an applicant who will not pay, carries a cost of 5, while a false positive, rejecting a creditworthy applicant, carries a cost of only 1. The bank loses far more money from a defaulted loan than from a missed business opportunity. Any improvement strategy must therefore prioritize reducing false negatives, even at the cost of more false positives.

Implementing Cost-Sensitive Weights

Both Logistic Regression and LinearSVC support a class_weight parameter that adjusts the penalty for misclassifying each class during training. Setting class_weight={0: 1, 1: 3} tells the model that every bad-risk misclassification (false negative) is 3 times more costly than a good-risk misclassification (false positive). A weight of 3 was chosen as a pragmatic compromise: it is more aggressive than the frequency-based balanced weighting and better reflects the domain cost structure, while avoiding the extreme false positive rate that the full 5:1 ratio would produce. kNN does not support this parameter and was left unchanged.

The full domain-specified weight of 5:1 was also tested. While it does reduce the number of missed bad-risk applicants from 10 to 7, recall remains unchanged at 0.833, meaning the additional false negatives avoided do not translate into a measurable recall improvement at this level of rounding. At the same time, the overall metrics worsen: precision drops further and F1 decreases, as the model becomes overly aggressive in flagging applicants as bad risk, producing too many false positives to justify the marginal gain. The 3:1 weight therefore represents the better trade-off for the purpose of this project, since it achieves the same recall while maintaining stronger F1 and fewer unnecessary rejections of good customers.

In practice, this means that during training, every time the model misclassifies a bad-risk applicant, the gradient update is 3 times larger than it would be for misclassifying a good-risk applicant. The model is therefore pushed to shift its decision boundary to be more cautious, flagging more applicants as bad risk to avoid costly false negatives. Nevertheless, it should be noted that the domain-specified cost ratio is 5:1, and on a larger dataset where the primary objective is to maximally identify bad-risk applicants, and where the cost of false positives is considered truly negligible, the full 5:1 weighting may be more appropriate.

Results After Improvement

Model	Train Accuracy	Test Accuracy	Precision	Recall	F1-Score	AUC
LR — Original	0.784	0.785	0.681	0.533	0.598	0.81
LR — Weighted (3:1)	0.720	0.720	0.521	0.833	0.641	0.81
SVM — Original	0.788	0.775	0.642	0.567	0.602	0.80
SVM — Weighted (3:1)	0.715	0.715	0.516	0.833	0.637	0.81

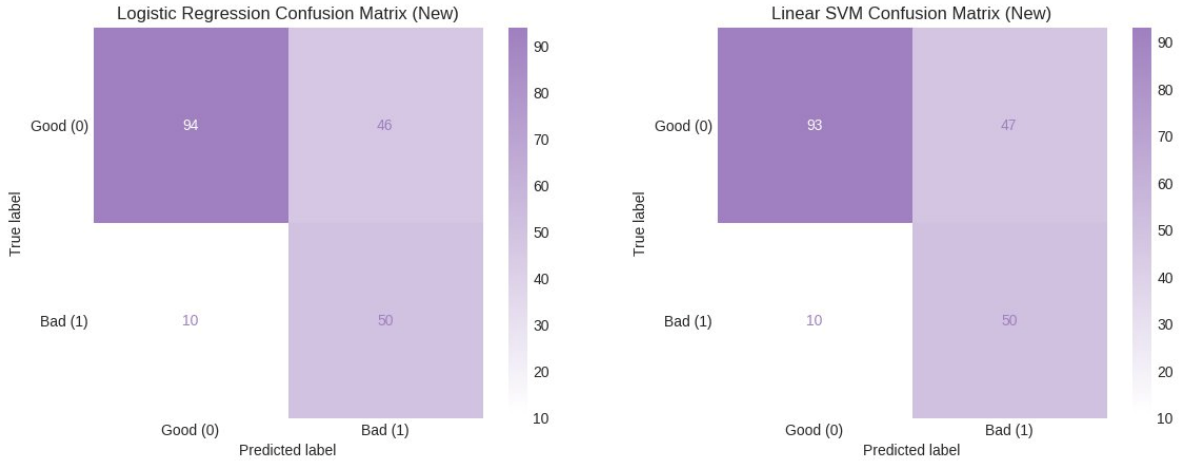


Figure 11. Confusion matrices after applying cost-sensitive weighting (3:1). Left: Logistic Regression (50/60 bad-risk correctly identified). Right: Linear SVM (50/60 bad-risk correctly identified).

Both models now correctly identify 50 out of 60 bad-risk applicants (recall = 0.833), up from 32 and 34 respectively in the baseline. This is a dramatic improvement in recall and directly addresses the cost structure of the problem, since only 10 bad-risk applicants are now missed, compared to 28 and 26 before. The tradeoff is an increase in false positives: Logistic Regression incorrectly flags 46 good-risk applicants as bad (up from 15), and SVM flags 47 (up from 19). Under the 3:1 cost weighting, these additional false positives are acceptable: each costs 1, while each avoided false negative saves 3.

F1-score improves for both models: LR 0.598 → 0.641 (+0.043), SVM 0.602 → 0.637 (+0.035). These are the strongest F1 results achieved across all experiments. The large recall gain (e.g.: 0.533 → 0.833 for Logistic Regression, with equivalent gains for SVM) outweighs the precision loss (0.681 → 0.521) in the harmonic mean, confirming that the cost-sensitive approach is genuinely beneficial. Test accuracy drops (0.785 → 0.720) but as established throughout this report, accuracy is not the appropriate metric on this imbalanced dataset.

The ROC curves confirm that AUC is unchanged by the cost weighting (LR: 0.81 → 0.81, SVM: 0.80 → 0.81). This is expected: adjusting class weights shifts the operating point on the ROC curve rather than changing the model's underlying discriminative ability. The improvement in F1 and recall is therefore a genuine gain in practical performance at the default threshold, not a change to the model's fundamental structure.

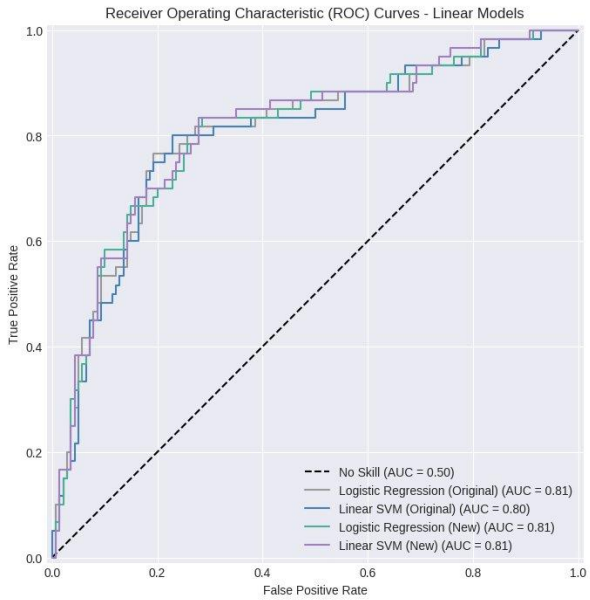


Figure 12. ROC curves comparing original vs. cost-weighted models.

While Section 7 recommended Linear SVM based on baseline F1, the introduction of cost-sensitive weighting in this section shifts the final recommendation to Logistic Regression. It achieves the highest F1 (0.641), substantially better recall than the original Logistic Regression (0.833 vs. 0.533), higher AUC (0.81), and is the model most aligned with the domain cost structure of the German Credit problem. Its calibrated probability outputs also allow the decision threshold to be tuned further if needed, and its coefficients remain fully interpretable for regulatory purposes.